

Actividad 26/06

Formato	Ventajas	Desventajas
CSV	Extremadamente ligero y fácil de generar desde Excel	No soporta jerarquías complejas, solo datos planos.
XML	Estructurado, soporta tipos de datos y jerarquías.	Verboso, archivos más pesados que JSON o CSV.

Diferencia técnica entre los procesos de Serialización y Deserialización utilizando la librería nativa System.Text.Json:

- **Serialización:** Es el proceso técnico mediante el cual un objeto fuertemente tipado en memoria activa de C#. Su propósito principal es la persistencia de datos (guardar en archivos) o el transporte a través de la red (enviar un payload hacia una API externa). En System.Text.Json, este flujo se ejecuta principalmente mediante el método estático JsonSerializer.Serialize().
- **Deserialización:** Es el proceso inverso. Consiste en tomar un flujo de texto plano o un string con formato estructurado JSON y parsear sintácticamente sus elementos para reconstruir y tipar dinámicamente un objeto equivalente en la memoria del entorno de ejecución de C#. Esto permite al desarrollador interactuar nuevamente con la información a través de propiedades y métodos nativos del lenguaje. En la librería nativa, se lleva a cabo mediante el método genérico JsonSerializer.Deserialize<T>(). Para garantizar que el mapeo sea exitoso aun cuando las propiedades del JSON vengan en minúsculas y las de C# en PascalCase, se debe instanciar la clase JsonSerializerOptions y configurar la propiedad PropertyNameCaseInsensitive = true.

2. El Antipatrón del Rendimiento: Problema "N+1" y Estrategia de Batching

¿En qué consiste el error N+1? Ocurre cuando el sistema procesa un archivo masivo de registros (como un CSV o XML) y ejecuta una operación de consulta o inserción individual en la base de datos por cada registro o línea que va leyendo en el ciclo de lectura. Si el archivo contiene un volumen de N registros, el programa realizará N peticiones individuales de red e inserción a la base de datos, sumadas a la transacción o consulta inicial ($+1$). Esto genera una degradación crítica del rendimiento debido al cuello de botella provocado por la latencia acumulada de red, la saturación del pool de conexiones y el sobreesfuerzo del motor de base de datos al procesar miles de transacciones atómicas aisladas.

La estrategia a utilizar sería la estrategia de Optimización por Lotes (Batching) en donde se implementa el procesamiento por lotes o Batching. En lugar de abrir una transacción por cada registro, el archivo se lee de forma secuencial y asíncrona en memoria empleando flujos se realiza una única operación de inserción masiva en la base de datos utilizando el método `AddRange()` (o su equivalente en el ORM) y se envía un solo comando de confirmación global mediante `SaveChangesAsync()`. Esto reduce drásticamente las interacciones de red a un único viaje (o unos pocos bloques), optimizando el tiempo de respuesta de forma exponencial.

Referencias Bibliográficas:

Facultad de Ingeniería, USAC. (2026). Sesión 20: Integración de Datos. Consumo de APIs Externas y Carga Masiva (CSV/XML). Laboratorio del curso Introducción a la Programación y Computación 2. Guatemala.